



Faculty of Computers and Artificial Intelligence

Cairo University

CS213: Object Oriented Programming
Under Supervision of Dr. Mohammed El-Ramly



جامعة القاهرة

Assignment: 3

Section Group: 23

Task: 1, 2 & 3

Gameboards – Assignment 3 Document

Name	Nada El Mazloun	Sondos Osama	Nourhan Ali	Mariam Mohamed
ID's	20242369	20240252	20240643	20240580
Emails	20242369@stud.fci-cu.edu.eg	20240252@stud.fci-cu.edu.eg	20240643@stud.fci-cu.edu.eg	20240580@stud.fci-cu.edu.eg

work breakdown – Assignment 3:

Name	Task 1	Task 2	Group Task	Bonus
Nada	4x4 Tic-tac-toe	Pyramid Tic-tac-toe	All Group Games Done Together	Memory Tic-tac-toe + AI
Sondos	SUS	4 in a row		Ultimate Tic-tac-toe + AI
Nourhan	Misère Tic-tac-toe	Diamond Tic-tac-toe		Memory Tic-tac-toe + AI
Mariam	5x5 Tic-tac-toe	Word Tic-tac-toe		Ultimate Tic-tac-toe

Classes Description:

1. *Sus*:

- ***Class:*** SUS and SUS_UI
- ***Description:*** SUS class represents a 3x3 board game where the objective is to form the sequence S-U-S (horizontally, vertically, or diagonally). The game is won not by the first player to form a sequence, but by the player who has the highest number of S-U-S sequences when the board is completely filled. SUS_UI class handles the specific User Interface required for the SUS Game, including the welcome message and standard input for move coordinates.
- ***Key Features:*** Maintains internal scores for Player 1 (score_p1) and Player 2 (score_p2). When a move is made, it calculates the increase in S-U-S sequences created by that move and adds the difference to the corresponding player's score. A player wins if their final score is strictly greater than the opponent's score. The game is a draw if the board is full and the scores are equal. Implements a helper function, count_SUS (),int get_score_s(), int get_score_u(), void display_scores() are functions to display the score and to find all possible "S-U-S" sequences on the board.

2. *Four in a row*:

- ***Class:*** Connect4_Board, Connect4_Player and Connect4_UI
- ***Description:*** Connect4_Board class represents a 6x7 Connect 4 game board. It implements the specific logic for piece dropping and the 4-in-a-row winning condition. Connect4_Player class primarily used by Connect4_UI during player creation. Connect4_UI class manages user interface and specific input requirements.
- ***Key Features:*** update_board method implements the core Connect 4 mechanic by making the move fall to the lowest available row in the chosen column. Win Condition is 4 consecutive symbols (horizontal, vertical, or diagonal). The game ends upon a win or when all 42 cells are filled without a win (a draw).

3. 5x5 Tic-tac-toe:

- **Class:** FiveByFive and FiveByFive_UI
- **Description:** FiveByFive class represents a specialized 5x5 Tic-Tac-Toe game board. It inherits from a generic Board<char> class and implements the specific logic for a larger board where the winning condition is based on the count of 3 in a row sequence after all cells are filled. UI class serves as the User Interface component for the 5x5 Tic-Tac-Toe game. It inherits from a generic UI<char> base class and handles the player interaction specifics for this game variant, such as showing the welcome message, creating players, and getting move input.
- **Key Features:** The game is considered over when 24 moves have been made. The game_is_over function returns true when n_moves == 24. The winning condition is based on a sequence count comparison between players only after 24 moves. The game is a draw if both players have an equal number of 3-in-a-row sequences. Sequence Counting: Implements a private helper function, count_sequences (char sym), to check for all possible 3 in a row sequence for a given symbol across the 5x5 board.

4. Word Tic-tac-toe:

- **Class:** Word_TicTacToe and Word_TicTacToe_UI
- **Description:** Word_TicTacToe class represents a specialized 3x3 Tic-Tac-Toe board where the winning condition is based on forming a valid 3-letter word in any row, column, or diagonal. It uses a dictionary to check the valid words. UI class handles the specific User Interface and input logic required for the Word Tic-Tac-Toe game, including prompting for a letter along with the coordinates.
- **Key Features:** Players place a single letter (char), Maintains and loads a dictionary of valid 3-letter words. The game ends either when a valid word is formed (is_win) or when the board is full and with no valid words formed (is_draw). The get_move function prompts the human player to enter the row, column, and the letter they want and provides a simple random move for the computer player.

5. *Misère Tic-Tac-Toe:*

- ***Class:*** miser
- ***Description:*** This game inverts the traditional rules. The player who forms 3-in-a-row loses instead of wins.
- ***Key Features:*** The `is_win ()` function always returns false. Instead, the game logic relies heavily on `is_lose ()` to detect if a player has accidentally formed a line.

6. *Diamond Tic-Tac-Toe:*

- ***Class:*** DiamondTicTacToe
- ***Description:*** This class inherits from the Board class. It implements a 5x5 grid where only specific cells form a diamond shape. The logic includes checking for patterns (X or O) specifically within the valid diamond indices.
- ***Key Features:*** Overrides `inside_diamond ()` to validate moves and uses a custom `is_win ()` to check diagonal and linear winning conditions inside the diamond shape.

7. *4x4 Tic-Tac-Toe:*

- ***Class:*** FourxFour_Board, FourxFour_Move and Fourxfour_UI
- ***Description:*** FourxFour_Board class manages all game logic board. Handles game state, rules, win checking and AI logic. FourxFour_Move class represents a single move in the game, including old position, new position and the symbol being moved. Fourxfour_UI class handles all user interaction, including reading input and displaying prompts. Connects the players to the board logic.
- ***Key Features:*** FourxFour_Board class initialize starting board layout , validates positions (in bounds()) , checks adjacency for moves (is_adjacent()) , updates board after moves (update_board()), detects win states (is_win ()) and draw states (is_draw()) , generates all valid moves for a player (get_all_valid_moves()) , implements minimax + alpha-beta pruning , evaluates board using evaluation1 and evaluation2. FourxFour_Move class stores move coordinates, store player symbol (X/O), transfers move data between UI, board logic and AI, encapsulates all move-related information cleanly. Fourxfour_UI class creates human or computer players, validates human input, requests AI moves using best_moves (), displays messages and errors to the user.

8. Pyramid Tic-Tac-Toe:

- ***Class:*** pyramidTicTacToe_board, pyramid_AI_player and a UI class
- ***Description:*** pyramidTicTacToe_board class represents the pyramid-shaped game board and contains all game logic related to valid moves, updating the board, win detection, and AI support. pyramid_AI_player class represents the AI player in the game and works alongside the board's AI logic. UI class handles all user interactions: player creation, move input, and the board.
- ***Key Features:*** pyramidTicTacToe_board class stores the pyramid as a padded 3*5 matrix, custom in_bounds () to match pyramid shape, applies moves with update_board(), detects wins using predefined pyramid winning lines, provides helper methods: empty_cells(); is_valid_move(); is_full(), implements AI supports : evaluate(); minimax(); best_move(), fully inherits from board<char> and overrides behavior where necessary. pyramid_AI_player class Inherits from Player<char>, Works with best_move() from the board to choose optimal moves, Holds the identity of the computer player, Simplifies integrating Minimax AI into the framework. UI class inherits from UI<char>, creates human and AI players via create_player(), Reads and validates human moves, uses dynamic_cast() to safely access the pyramid board, calls best_move() to get the AI's decision, converts input into move<char> objects.

9. Numerical Tic-Tac-Toe:

- ***Class:*** NumericalBoard
- ***Description:*** A variation where players use numbers instead of symbols. Player 1 uses odd numbers, and Player 2 uses even numbers.
- ***Key Features:*** The is_win () function checks if any row, column, or diagonal sums up to exactly 15. It also uses a std::set to track used numbers to prevent repetition.

10. Obstacles Tic-Tac-Toe:

- ***Class:*** Obstacles_XO
- ***Description:*** class extends the generic Board class to create a more challenging, larger scale. It operates on a 6x6 grid. The core mechanic involves the introduction of "Obstacles" (using #) as block cells, preventing players from using them. The class manages the generation of these obstacles and enforces a modified winning condition suitable for the larger board size.
- ***Key Features:*** 1.the is_win() function is overridden to check for 4 consecutive symbols (horizontally, vertically, or diagonally) instead of 3 2.add_obstacles) to randomly place non-playable symbols (#) on the board.This forces players to play inside this cell 3.The constructor initializes the random number generator (srand), ensuring that if obstacles are placed randomly

11. Infinity Tic-Tac-Toe:

- ***Class:*** Infinity_XO
- ***Description:*** This class inherits from the Board class. It implements a 3x3 grid where moves are temporary. The logic employs a First-In-First-Out (FIFO) system where the oldest mark on the board is automatically removed after every three moves.
- ***Key Features:*** Utilizes a std::queue data structure to track the history of moves. Overrides update_board() to implement the logic, removing the oldest cell from the board when the total move count is a multiple of three.

12. Ultimate Tic-Tac-Toe:

- **Class:** UltimateMove, UltimateBoard, UltimatePlayer, UltimateComputerPlayer and Ultimate_UI
- **Description:** UltimateMove class represents a specific move within the Ultimate XO game. It extends the base Move<char> but tracks of the main board position and the sub-board position of the chosen cell. UltimateBoard class Implements the board structure and core game rules. It manages a 3x3 array of 3x3 boards and a separate 3x3 main board to track the sub-board winners. UltimateComputerPlayer class represents an AI opponent in the Ultimate Tic-Tac-Toe game, inheriting from UltimatePlayer. Ultimate_UI class Manages the user interface, including setting up players, gathering input, and displaying the complex, multi-layered board state.
- **Key Features:** The move class stores main_row, main_col, sub_row, and sub_col. It is cast from the generic Move<char>* pointer to retrieve all four coordinates when updating the board state. Board class Enforces the crucial rule where the sub_row and sub_col of the last move determine the mandatory main_row and main_col for the next move. It overrides the base class functions to check for a win condition on the main board using check_win_condition(main_board, symbol). Player class Used as the base for human and computer players, allowing the game manager to treat all players polymorphically via the Player<char>* interface. Computer player class Its type is set to PlayerType::COMPUTER, which signals the Ultimate_UI to execute the random move generation logic in get_computer_move. UI class Contains get_human_move() to prompt the user for four coordinates and get_computer_move() to generate a valid, random move from the list of all possible moves (get_valid_moves). It is responsible for calling display_ultimate_board().

13. Memory Tic-Tac-Toe:

- **Class:** Memory_XO
- **Description:** A memory-based game where moves are hidden after being played. The board displays placeholders (questions marks), and the actual symbols ('X' or 'O') are stored in a private hiddenboard array.
- **Key Features:** The logic involves mapping visible moves to the hidden board and momentarily revealing symbols or checking matches blindly.

“A PIE” OOP 4 principles:

1. *Abstraction:*

- **Concept:** Hiding complex implementation details and showing only the necessary features of an object.
- **Applied:** We created generic abstract base classes (Board and Player) in BoardGame_Classes.h. These classes define the general rules (update_board, is_win) without specifying how they work. The GameManager interacts with these abstract interfaces, allowing it to run any game without knowing its specific logic.

2. *Inheritance:*

- **Concept:** Creating new classes from existing ones to reuse code.
- **Applied:** All our specific games inherit from the base Board class.
 - DiamondTicTacToe inherits from Board<char>.
 - NumericalBoard inherits from Board<int>.
 - Memory_XO inherits from Board<char>.
 - miser inherits from Board<char>. This allowed us to reuse common attributes like rows, columns, and n_moves without rewriting them for every game.
 - FiveByFive class inherits from Board<char>. Class FiveByFive_UI inherits the core UI structure.

- Word_TicTacToe class inherits from Board<char>. Class Word_TicTacToe_UI inherits from UI<char>.
- SUS class inherits from Board<char>. SUS_UI class inherits from UI<char>
- Connect4_Board inherits from Board<char>, Connect4_Player inherits from Player<char> and Connect4_UI inherits from UI<char>.
- Infinity_XO inherits properties like rows, columns, n_moves, and the 2D board array from the parent class.
- pyramidTicTacToe_board inherits from Board<int>.
- pyramid_tic_tac_toe_UI inherits from UI<char>
- Pyramid_AI_Player inherits from Player<char>
- FourxFour_Board inherits from Board<char>
- FourxFour_Move inherits from Move<char>
- FourxFour_UI inherits from UI<char>
- Player<char> is reused from the base framework.
- Obstacles_XO inherits from Board<char>.
- Ultimate_XO inherits from the base Move<char>, inherits from the base Board<char>, Inherits from the base Player<char>, Inherits from UltimatePlayer which in turn inherits from Player<char>.

3. *Polymorphism:*

- **Concept:** The ability of different classes to be treated as instances of the same general class through Virtual Functions.
- **Applied:** We used Runtime Polymorphism via virtual functions (update_board, is_win). Although the function name is the same, the behavior changes based on the object type:
 - In DiamondTicTacToe: is_win checks for diagonal lines within the diamond shape.
 - In Numerical Tic-tac-toe: is_win checks if the sum of a line equals 15.
 - In Misere Tic-tac-toe: The logic is inverted; is_win returns false, and we check is_lose instead.
 - In Memory XO: update_board hides the symbol after playing.

- In 5x5 tic-tac-toe: `bool is_win (Player<char> *player)` checks sequence counts after the board is full (`n_moves < 24`), not for an immediate 3 in a row.
- In Word tic-tac-toe: `is_win (Player* p)` implements the complex process of checking for 8 possible 3-letter combinations (rows, columns, diagonals) on the board and then calling `is_valid_word ()` on each combination.
- In SUS: `is_win (Player* player) / is_draw (Player* player)` implement the specific win/draw condition of the SUS game by comparing the final scores (`score\_p1, score\_p2`) after all cells are filled.
- In connect 4: `is_win (Player* p)` implement the 4-in-a-row condition across the 6x7 grid. `Get_move (Player* p)` specializes input, asking player only for a column.
- In Infinity_XO: `is_draw ()` to always return false
- In pyramid tic tac toe: Several functions are overridden to customize behavior: `update_board()`, `is_win ()`, `is_draw()`, `game_is_over()`, `get_move()` in the UI, `create_player()`.
- In 4*4 tic tac toe: Virtual functions from the base classes: `update_board`, `is_win`, `game_is_over`, and UI functions are overridden to implement rules specific to the 4x4 version.
- In 4*4 tic tac toe: AI and human players both use the same `get_move()` interface: Human move is read from console, computer move is generated by minimax, they both return a `Move<char>*`.
- In obstacles XO: `is_win()`: The Obstacles_XO version creates a new algorithm to scan a 6x6 grid for 4-in-a-row patterns. `update_board()`: To handle the specific logic of placing marks on this larger grid.
- In ultimate: It overrides virtual functions like `update_board()`, `is_win()` and `is_draw()` to implement the complex Ultimate Tic-Tac-Toe logic. The generic `get_move(Player<char>* player)` method dispatches the call internally to either `get_human_move` or `get_computer_move` based on the player's runtime type (type checking).

4. *Encapsulation:*

- **Concept:** Bundling data and methods that operate on that data within a single unit and restricting access to some components.
- **Applied:**
 - The game grid (board [] []) is declared as protected in the base class, so it can only be modified by the derived game classes, not by the main function.
 - In Memory XO: The hiddenboard array is private, ensuring that no external code can cheat and see the hidden values.
 - In 5x5 tic-tac-toe: The sequence checking function count_sequences (char sym) a private method that encapsulates the win logic details, ensuring external code only interacts with the simple boolean interface is_win ().
 - In word tic-tac-toe: The dictionary is like a static private member and is only accessed through the helper function is_valid_word () and load_dictionary (). The update_board (Move* move) controls how a letter is placed. It encapsulates validation logic (checking boundaries and cell occupancy), ensuring the board state is only modified correctly and consistently.
 - In SUS: The player scores (score_p1 and score_p2) are declared as private members within the SUS class. The count_SUS () function is an internal function of the SUS class that prevents external classes from directly accessing the score mechanism.
 - In connect 4: Checking all possible 4-in-a-row sequences is encapsulated within the is_win () function.
 - In Infinity_XO: The vector<pair<int, int>> (or queue) used to track the history of moves is declared as private / The board matrix is modified only through the public update_board () method, ensuring that the synchronization between the board state and the history queue is always maintained correctly.
 - In Pyramid tic tac toe: Board layout is stored internally and only accessed through getters/setters, blank_symbol is private, move validation is controlled through in_bounds() and is_valid_move().

- In 4*4 tic tac toe: Each class hides its internal details: In Board class: The board matrix is protected inside Board<char>, Access is controlled through member functions like: get_board_matrix() update_board(). In move calss: Coordinates are stored privately, Access only through getters like getNewX() and getNewY(). In UI class: Input-handling logic is fully separated from the game logic, Internal data cannot be modified directly, all changes go through controlled interfaces.
- In obstacles: The vector<pair<int, int>> obstacles and the obstacle_symbol (#) are declared as private.
- In ultimate: The coordinates (main_row, etc.) are held as private data members and accessed via public getter methods (e.g., get_main_row()). The 4D ultimate_board and 3x3 main_board data structures are kept private, with updates strictly controlled by the public update_board() and update_main_board() methods.

Quality of the code:

1. Misère Tic Tac Toe:

- 1.Critical Logic Error: is_win() always returns false, This is wrong because in Misère Tic Tac Toe, winning = not losing.
Here she must handle "the opponent lost".
- 2.set_cell() incorrectly increments/decrements moves : code : (if (symbol != 0) n_moves++; else n_moves--) , This is dangerous: AI repeatedly places/removes cells in minimax, When resetting a cell (set_cell(x, y, 0)), it decreases n_moves even for cells that were never counted. This will corrupt draw detection.
Correct way: Only increment n_moves when updating empty -> non-empty.
- 3.Game End Condition Incorrect: code: (return is_lose(p) || is_draw(p);), This declares game over only for current player. Correct Misère rule:
If player A plays and forms 3-in-a-row → A loses immediately,
You don't check opponent losing conditions when switching turns.

4. Wrong use of Player inside Minimax: AI creates: (Player dummy("dummy", dummyplayer, PlayerType::HUMAN);). This dummy object does not reflect real opponent state (symbol only). More importantly, minimax shouldn't rely on UI class or Player objects at all. AI shouldn't depend on high-level classes → breaks OOP layering.

2. *Pyramid tic tac toe:*

function is _winner exists but is never used, so it can be removed. Also, there are several magic numbers like 9 and 1000 that should be replaced with named constants to make the code easier to understand. In addition, short variable names such as r and c can be changed into clear names such as row and col. Overall, the game works well, but these small changes would make the implementation cleaner and more readable.

3. *Word XO:*

1. Removed Dead Code: Deleted unused commented-out code at the end of the header file.
2. Input Validation: Verified that the game checks if the input is a letter (A-Z) using isalpha.
3. File Safety: Checked that the game handles errors if dic.txt is missing.

4. *Sus:*

The game is generally correct but contains a confusing game design point: Redundant AI Fallback: The SUS_AI_Player::get_move () function ends with a fallback move: return new Move<char> (0, 0, symbol); // fallback. Issue: The preceding loops check every cell, meaning this fallback should never be reached if there is an empty spot. If the board is full, the game should have ended before this function was called. Recommendation: This line can be safely removed.

Video link:

https://drive.google.com/file/d/1U6e0Ex_LSZPGjbghiM56qVut80n3EuqE/view?usp=drive_link

Screenshots of the program and repo:

```
Computer Mariam chooses: (0, 0) with letter 'L'

      0   1   2
-----
0 | L | R | U |
-----
1 | L | X | X |
-----
2 | G | P | Q |
-----

Draw!
```

```
Computer Nada chooses: (0, 2)
This cell is already occupied!
Computer Nada chooses: (4, 3)
This cell is blocked by an obstacle!
Computer Nada chooses: (1, 3)

      0     1     2     3     4     5
-----
0 |   # |   # |   X |   # |   # |   0 |
-----
1 |   X |   # |   X |   0 |   # |   # |
-----
2 |   # |   # |   # |   # |   # |   # |
-----
3 |   # |   0 |   0 |   X |   0 |   0 |
-----
4 |   # |   X |   X |   # |   X |   # |
-----
5 |   0 |   # |   X |   0 |   X |   0 |
-----

Draw!
```

```
      0   1   2   3   4
-----
0 |   |   | X |   |   |
-----
1 |   | 0 | 0 | . |   |
-----
2 | . | 0 | X | . | X |
-----

Computer chose: 1, 3

      0   1   2   3   4
-----
0 |   |   | X |   |   |
-----
1 |   | 0 | 0 | X |   |
-----
2 | . | 0 | X | . | X |
-----

Nada wins!
```

```
Mariam (S) is thinking...

SCORES
S: 0 points
U: 1 points

      0   1   2
-----
0 | U | S | S |
-----
1 | U | S | U |
-----
2 | U | S | S |
-----

Marioum wins!

=== GAME OVER ===
```


Commits on Nov 28, 2025

Edited Word TicTacToe	0e783ae		
mariam-mohamed225 committed last week			
Added Word TicTacToe	6052e9e		
mariam-mohamed225 committed 2 weeks ago			

Commits on Nov 27, 2025

Edited Obstacles_XO	dd71bd2		
mariam-mohamed225 committed 2 weeks ago			
deleting comments	0b8f355		
sondososama70 committed 2 weeks ago			
edit_obstacles_group_game	0fda14b		
NourhanAli-fcai28 committed 2 weeks ago			
Continue The Obstacles_XO	0107b29		
mariam-mohamed225 committed 2 weeks ago			
Corrected 5x5_XO	5b3b709		
mariam-mohamed225 committed 2 weeks ago			
adding win function to obstacles_XO	213ad98		
nadaahmed950 committed 2 weeks ago			

Commits on Dec 8, 2025

Added Ultimate Game	fd2614e		
mariam-mohamed225 committed 40 minutes ago			
...	bb7a386		
mariam-mohamed225 committed 1 hour ago			
Added Html doxygen comments	f36968c		
mariam-mohamed225 committed 2 hours ago			
AI SUS game	c7c658a		
sondososama70 committed 2 hours ago			

Commits on Dec 7, 2025

Merge branch 'main' of https://github.com/sondososama70/CS_Oop_ASS3_boardgame	cacc210		
NourhanAli-fcai28 committed yesterday			
fix infinity & include memory	98ce19e		
NourhanAli-fcai28 committed yesterday			
Update main.cpp	844ae5a	Verified	
NourhanAli-fcai28 authored yesterday			
Update main.cpp(add game 11 & 13 to menu)	e0472f7	Verified	
NourhanAli-fcai28 authored yesterday			
fix DiamondTicTacToe.h	a5ed0ba	Verified	
NourhanAli-fcai28 authored yesterday			

CS_Oop_ASS3_boardgame

Private

Watch 0 Fork 0 Star 0

main 1 Branch 0 Tags

Go to file

Add file Code

mariam-mohamed225

Added Ultimate Game

fd2614e · 39 minutes ago

64 Commits

.idea	AI SUS game	2 hours ago
.vs	yaaarb	2 weeks ago
cmake-build-debug	Added Ultimate Game	39 minutes ago
out/build/x64-Debug	yaaarb	2 weeks ago
.gitattributes	Initial commit	2 months ago
BoardGame_Classes.h	editing sus	2 weeks ago
CMakeLists.txt	AI for four x four tic tac toe and pyramid tic tac toe	2 days ago
Connect4.cpp	AI for four x four tic tac toe and pyramid tic tac toe	2 days ago
Connect4.h	Added Html doxygen comments	2 hours ago
DiamondTicTacToe.cpp	fix DiamondTicTacToe.cpp	yesterday
DiamondTicTacToe.h	fix DiamondTicTacToe.h	yesterday
FiveByFive.cpp	Corrected 5x5_XO	2 weeks ago
FiveByFive.h	Added Html doxygen comments	2 hours ago

About

No description, website, or topics provided.

Activity

0 stars

0 watching

0 forks

Releases

No releases published

Create a new release

Packages

No packages published

Publish your first package

Contributors 4

sondososama70

mariam-mohamed225

Mariam_Moha...

NourhanAli-fcai28

nadaahmed950